Obligatorisk oppgave om Titanic

HON2200

Ådne Rolstad

18. mars 2026

Introduksjon

I denne obligatoriske oppgaven undersøker jeg hvordan ulike modeller kan brukes til forstå hvilke egenskaper som påvirker hvem som overlevde Titanic-forliset.

I Oppgave 1 blir data lest inn som pandas Dataframes, manglende verdier blir håndtert, kategoriske variabler blir one hot-encodet og data blir delt inn i treningsdata og valideringsdata. I oppgaveteksten er det informert om at testdata allerede er fjernet fra datasettet, og dette er grunnen til at jeg konsekvent bruker begrepet "valideringsdata" i oppgaveteksten og i koden.

I Oppgave 2 plotter jeg sigmoid-funksjonen og implementerer logistisk klassifikasjon. Koden jeg har brukt for å teste implementeringen kan finnes i filen `testing.ipynb` i Github: `https://github.com/aadnerol/HON2200/tree/main/oblig/code`. Denne implementeringen blir så testet på de dataene vi jobber med.

I Oppgave 3 undersøker vi beslutningstrær. Først blir et tre med maksdybde 3 tilpasset dataene, før vi undersøker hvordan økende maksdybde påvirker prediksjonene.

I Oppgave 4 lager vi en random forest-modell, og lager figurer som viser feature importance og partial dependence for parameterne. Til slutt diskuterer vi om man kan forutsi hva modellen vil predikere for de ulike typene modeller.

Oppgave 1

1a

For å lese datasettet `train.csv` med pandas, brukte jeg følgende kode i Python:

```python
import numpy as np
import pandas as pd

path = "../data/train.csv"
df = pd.read_csv(path, header=0)
df.head()
df = df.set_index("PassengerId")
print(f"Antall observasjoner: {len(df)}")
df.head()
```

Listing 1: Lesing av datasett

Denne koden lastet inn dataene og visualiserte de 5 første kolonnene i datasettet. Kolonnene i datasettet er:

- `Survived`: Kategorisk variabel som beskriver om personen overlevde eller ikke.
- `Pclass`: Billettklasse til personen. 1 er best, 3 er dårligst.
- `Name`: Navnet til personen.
- `Sex`: Kjønnet til personen.
- `Age`: Alder til personen.
- `SibSp`: Antall søsken og/eller partnere personen hadde om bord skipet.
- `Parch`: Antall foreldre og/eller barn personen hadde om bord skipet.
- `Ticket`: Billettnummeret til personen.
- `Fare`: Prisen personen betalte for billetten.
- `Cabin`: Hvilen kabin personen hadde.
- `Embarked`: Hvilken havn personen gikk om bord skipet. C står for Cherbourg, Q står for Queenstown og S står for Southampton.

1b

Får å håndtere manglende data for alder, kjørte jeg først den følgende koden for å få oversikt over hvilke data som manglet:

```python
display(df.isnull().sum())
```

Listing 2: Oversikt over manglende data

Denne koden ga følgende output:

```
Survived      0
Pclass        0
Name          0
Sex           0
Age         177
```

```
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin       687
Embarked      2
dtype: int64
```

Kolonnen `Age` har dermed 177 rader med manglende data. Siden det er relativt få rader i datasettet, utgjør dette en stor andel. Vi lager dermed to mulige Dataframes. I det ene Dataframet blir radene med manglende verdier fjernet. I det andre blir manglende verdier erstattet med den gjennomsnittlige alderen til personer som har samme kjønn og billettklasse. Det er det sistnevnte datasettet jeg bruker videre i analysen. Koden under beskriver hvordan dataframet ble konstruert:

```python
df_remove_nan = df.dropna(subset="Age")

mean = np.mean(df_remove_nan["Age"]).round()
# print(mean)
df_mean_nan = df.copy()
df_mean_nan["Age"] = df_mean_nan.groupby\
    (["Pclass", "Sex"],group_keys=False)["Age"].apply(lambda x: x.fillna(x.median()))
```

Listing 3: Håndtering av manglende verdier.

1c

I koden under blir kolonnene `Sex`, `Embarked` og `Pclass` transformert ved hjelp av one hot encoding:

```python
df = df_mean_nan
categorical_cols = ["Pclass", "Sex", "Embarked"]
df = pd.get_dummies(df, columns=categorical_cols, prefix=categorical_cols)
# display(df.head())
```

Listing 4: One hot encoding av kategoriske data.

1d

Følgende kolonner ble brukt til å predikere overlevelsen til en passasjer:

- `Age`
- `SibSp`
- `Parch`
- `Fare`
- `Pclass_1`
- `Pclass_2`
- `Pclass_3`
- `Sex_male`
- `Sex_female`
- `Embarked_C`
- `Embarked_Q`

- `Embarked_S`

Det kan diskuteres om det er nødvendig å inkludere både prisen en passasjer betalte for billetten og hvilken billettklasse personen hadde, da disse mest sannsynlig er veldig korrelert. Likevel inkluderte jeg begge i analysen videre. `Pclass_1`, `Pclass_2` og `Pclass_3` er de one hot encodede variablene for billettklasse, `Sex_male` og `Sex_female` er de one hot encodede variablene for kjønn, og `Embarked_C`, `Embarked_Q` og `Embarked_S` er de one hot encodede variablene for påstigninghavn.

Koden for å konstruere target-vektoren **y** og designmatrisen **X** er gitt under:

```
y_df = df["Survived"]
covariates = ["Age",
              "SibSp",
              "Parch",
              "Fare",
              "Pclass_1",
              "Pclass_2",
              "Pclass_3",
              "Sex_male",
              "Sex_female",
              "Embarked_C",
              "Embarked_Q",
              "Embarked_S"]
X_df = df[covariates]
# Gjør om til numpy arrays:
y = y_df.to_numpy()
X = X_df.to_numpy()
```

Listing 5: Konstruering av designmatrise og target.

Jeg transformerte dataframene til numpy-arrays, da jeg er mer vant til å jobbe med disse.

1e

I oppgaveteksten er det spesifisert at test-data allerede er fjernet og holdt unna datasettet. Dermed deler vi datasettet inn i treningsdata og valideringsdata. I oppgaven valgte jeg å bruke 80% av dataene til trening, og 20% av dataene til validering. Koden under viser hvordan dataene blir splittet:

```
from sklearn.model_selection import train_test_split
seed = 27042001
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=seed)
print(f"X_train shape: {np.shape(X_train)}")
print(f"X_val shape: {np.shape(X_val)}")
print(f"y_train shape: {np.shape(y_train)}")
print(f"y_val shape: {np.shape(y_val)}")
```

Listing 6: Konstruering av trenings- og valideringsdata.

Oppgave 2

2a

For å plotte sigmoidfunksjonen, lagde jeg først en funksjon i python som returnerer sigmoidfunksjonen:

```python
import numpy as np

def sigmoid(z: float) -> float:
    """Sigmoid-funksjonen. Tar høyde for overflow med np.where()

    Args:
        z (float): Input

    Returns:
        float: Funksjonsverdi
    """
    return np.where (z >= 0, 1 / (1 + np.exp(-z)), np.exp(z) / (1 + np.exp(z)))
```

Listing 7: Sigmoidfunksjonen i python.

For å plotte funksjonen, brukte jeg følgende kode:

```python
from methods import sigmoid
import matplotlib.pyplot as plt

z = np.linspace(-7, 7, 500)
plt.figure(figsize=(9, 3))
plt.plot(z, sigmoid(z))
plt.title("Sigmoid-funksjonen, f(z)")
plt.xlabel("z")
plt.ylabel("f(z)")
plt.savefig("../figures/sigmoid.pdf", format="pdf")
```

Listing 8: Plotting av sigmoidfunksjonen.

I figur 1 ser man plottet av sigmoidfunksjonen. Grunnen til at denne funksjonen egner seg til å svare på ja/nei-spørsmål, er at verdimengden til funksjonen er intervallet $(0, 1)$. Dermed kan man tolke funksjonsverdien som sannsynligheten for et utfall.

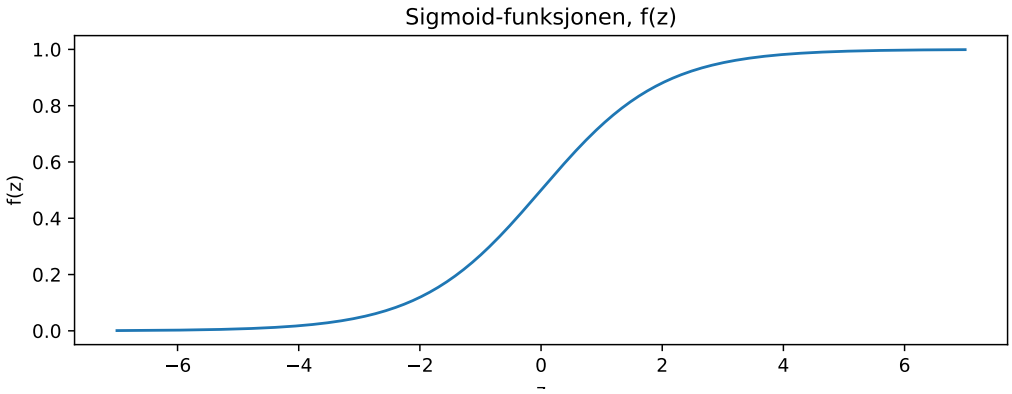

Figure 1: Sigmoidfunksjonen.

2b

Implementeringen av logistisk regresjon ble gjort i en separat python-fil, og koden kan finnes i Appendix. Hver funksjon i koden ble testet i en separat notebook, som kan finnes på `https://github.com/aadnerol/HON2200/blob/main/oblig/code/testing.ipynb`.

2c

For å teste implementeringen på dataene, brukte jeg koden under. Først tilpasset jeg modellen med treningsdataene:

```
from methods import fit_model
from sklearn.metrics import accuracy_score
from methods import output

X_train = np.asarray(X_train, dtype=float)
seed = 270401
eta = 0.001
N = 10000
W, b = fit_model(X_train, y_train, seed, eta, n_iters=N)
```

Listing 9: Tilpasse modell på treningsdata.

Deretter predikerte jeg på valideringsdataene, og beregnet accuracy score:

```
X_val = np.asarray(X_val, dtype=float)
y_hat = output(X_val, W, b)
y_hat = np.round(y_hat)

acc_model = accuracy_score(y_true=y_val.reshape(-1, 1), y_pred=y_hat)

# Model som antar at alle dør:
y_all = np.zeros(len(y_val))
acc_all = accuracy_score(y_true=y_val, y_pred=y_all)

print(f"Prediksjons-accuracy, logistisk regresjon: ")
print(f"Læringsrate: {eta}")
print(f"Antall iterasjoner: {N}")
print(f"Accuracy score: {np.round(acc_model, 4)}")
print(f"Accuracy score for modell som predikerer at ingen overlever: {np.round(acc_all, 4)}")
```

Listing 10: Prediksjon på valideringsdata og utregning av accuracy score.

Koden over ga følgende output:

```
Prediksjons-accuracy, logistisk regresjon:
Læringsrate: 0.001
Antall iterasjoner: 10000
Accuracy score: 0.7709
Accuracy score for modell som predikerer at ingen overlever: 0.6145
```

Vi ser at implementeringen av logistisk regresjon ga en treffprosent på omtrent 77%, som vil si at modellen predikerer riktig for omtrent 3 av 4 passasjerer.

Oppgave 3

3a

For å tilpasse et tre med maksdybde 3 som tilpasser treningsdataene, måtte jeg først få oversikt over dimensjonene på dataene jeg bruker:

```
from sklearn import tree

print(f"X_train shape: {X_train.shape}")
print(f"X_train datatype: {X_train.dtype}")
print(f"X_val shape: {X_val.shape}")
print(f"X_val datatype: {X_val.dtype}")
print(f"y_train shape: {y_train.shape}")
print(f"y_train datatype: {y_train.dtype}")
print(f"y_val shape: {y_val.shape}")
print(f"y_val datatype: {y_val.dtype}")
```

Listing 11: Undersøke dimensjoner.

For å få riktige dimensjoner på **y**, brukte jeg følgende kode:

```
y_train = y_train.reshape(-1, 1)
y_val = y_val.reshape(-1, 1)
print(f"y_train shape: {y_train.shape}")
print(f"y_train datatype: {y_train.dtype}")
print(f"y_val shape: {y_val.shape}")
print(f"y_val datatype: {y_val.dtype}")
```

Listing 12: Transformering av **y**

Til slutt tilpasset jeg et beslutningstre med maksdybde 3 med følgende kode:

```
tre = tree.DecisionTreeClassifier(max_depth=3)
tre = tre.fit(X_train, y_train)
```

Listing 13: Konstruksjon av et beslutningstre med maksdybde 3 som tilpasser treningsdataene.

3b

For å teste treet på valideringsdataene brukte jeg følgende kode:

```
y_hat=tre.predict(X_val)
acc_tre = accuracy_score(y_true=y_val, y_pred=y_hat)
print(f"Accuracy score, beslutningstre med maks dybde 3: {np.round(acc_tre, 4)}")
```

Listing 14: Testing av treet på valideringsdata.

Denne koden ga følgende output:

```
Accuracy score, beslutningstre med maks dybde 3: 0.8045
```

Altså hadde beslutningstreet en treffprosent på omtrent 80%, uten noen eksperimentering med parametre.

3c

Treet ble plottet med følgende kode:

```
plt.figure(figsize=(16,10))
tree.plot_tree(tre, max_depth=3, feature_names=X_df.columns, class_names=["Dead", "Alive"], filled=True)
plt.title("Beslutningstre med maksdybde 3")
plt.savefig("../figures/beslutningstre_3.pdf", format="pdf")
```

Listing 15: Plotting av treet med maksdybde 3.

Figur 2 viser plottet av beslutningstreet.

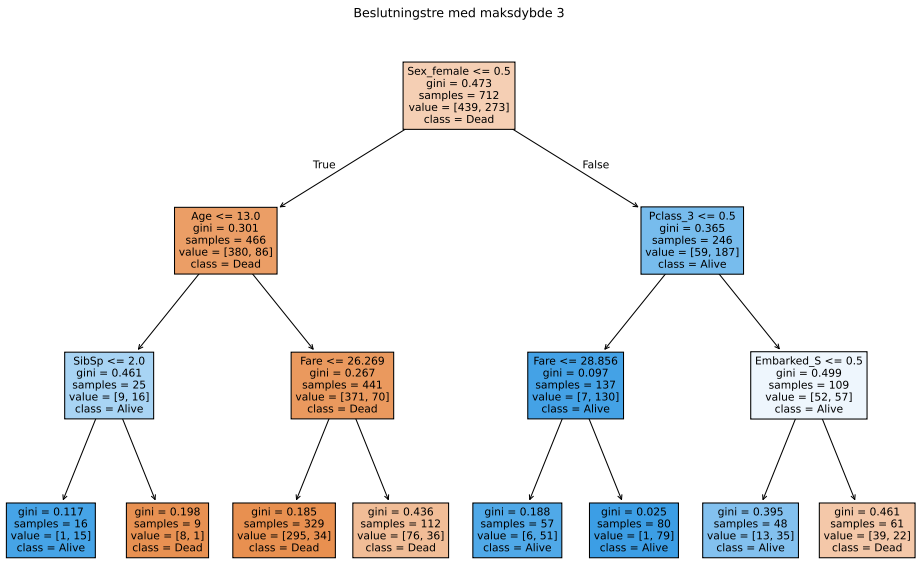

Figure 2: Beslutningstre med maksdybde 3.

Under følger en beskrivelse av hver forgreining i treet:

- **Nivå 1**:
  - **Node 1**: Treet sjekker om personen er mann eller kvinne.
- **Nivå 2**:
  - **Node 1**: Sjekker om personen har billettklasse 3.
  - **Node 2**: Sjekker om personen er under 13 år.
- **Nivå 3**:
  - **Node 1**: Sjekker om billetten til personen kostet under 28.856
  - **Node 2**: Sjekker om personen ikke steg ombord i Southampton
  - **Node 3**: Sjekker om personen hadde 2 eller færre søsken eller partnere om bord
  - **Node 4**: Sjekker om personen betalte mindre enn 26.269 for billetten.

Treet sier altså at følgende personer vil overleve:

- Kvinner med billettklasse 3, som ikke gikk på i Southampton
- Kvinner som ikke har billettklasse 3, som betalte mer enn 28.856 for billetten
- Kvinner som ikke har billettklasse 3, som betalte mindre nn 28.856 for billetten
- Menn under 13 som har færre enn 2 søsken eller partnere om bord.

3d

For å undersøke hvordan maksdybden til treet påvirker hvor godt treet klassifiserer testdataene, brukte jeg følgende kode:

```
maksybder = np.arange(1, 11, 1)
accuracies_val = []
accuracies_train = []

for dybde in maksybder:
    tre = tree.DecisionTreeClassifier(max_depth=dybde)
    tre = tre.fit(X_train, y_train)
    # Valideringsdata
    y_hat_val = tre.predict(X_val)
    acc_val = accuracy_score(y_true=y_val, y_pred=y_hat_val)
    accuracies_val.append(acc_val)
    # Treningsdata (sjekke overfitting)
    y_hat_train = tre.predict(X_train)
    acc_test = accuracy_score(y_true=y_train, y_pred=y_hat_train)
    accuracies_train.append(acc_test)

accuracies_val=np.round(accuracies_val, 4)
# print(accuracies)
for dybde in maksybder:
    print(f"Accuracy med maksdybde {dybde}: {accuracies_val[dybde-1]}")

```

Listing 16: Undersøking av hvordan maksdybde påvirker treffprosent.

Koden ga følgende output:

```
Accuracy med maksdybde 1: 0.7486
Accuracy med maksdybde 2: 0.7374
Accuracy med maksdybde 3: 0.8045
Accuracy med maksdybde 4: 0.8101
Accuracy med maksdybde 5: 0.8101
Accuracy med maksdybde 6: 0.8156
Accuracy med maksdybde 7: 0.8045
Accuracy med maksdybde 8: 0.7989
Accuracy med maksdybde 9: 0.7933
Accuracy med maksdybde 10: 0.7989
```

For å gjøre resultatene tydeligere, plottet jeg dem med følgende kode:

```
plt.figure(figsize=(9,3))
plt.plot(maksybder, accuracies_val,marker="o", label = "Valideringsdata")
plt.plot(maksybder, accuracies_train, marker="o", label="Treningsdata")
plt.title("Accuracy Score, trening og validering")
plt.ylabel("Accuracy score")
plt.xlabel("Beslutningstreets maksdybde")
plt.legend()
plt.savefig("../figures/acc_score_maksdybde_10.pdf", format="pdf")
```

Listing 17: Plotting av hvordan maksdybde påvirker klassifisering.

Figur 3 viser hvordan accuracy score påvirkes av maksdybden til treet, for treningsdata og valideringsdata. Man kan se at treffprosenten til treningsdataene øker, mens treffprosenten til valideringsdataene når et makspunkt. Dette kan være et tegn på overtilpasning ved maksdybder høyere enn 5.

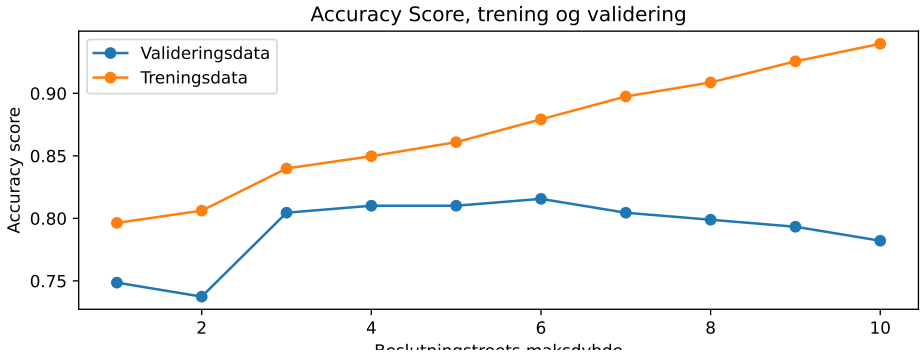

Figure 3: Maksdybde og accuracy score. Økende maksybde gir bedre resultat på treningsdata, men accuracy score flater ut for valideringsdata.

Oppgave 4

4a

For å gjøre det enklere å lage figurer senere i denne oppgaven, startet jeg med å gjøre dataene mine til pandas Dataframes i stedet for numpy arrays. Koden under splitter dataene med samme `seed` som tidligere:

```
seed = 27042001
X_train, X_val, y_train, y_val = train_test_split(X_df, y_df, test_size=0.2, random_state=seed)
```

Listing 18: Lagre data som pandas Dataframes

For å lage en random forest-modell som tilpasser treningsdataene, brukte jeg følgende kode:

```
from sklearn.ensemble import RandomForestClassifier

skog = RandomForestClassifier(n_estimators=100, max_features="sqrt", random_state=seed)
skog.fit(X_train, y_train.ravel())
print(f"Skog med {skog.n_estimators} trær, max_features: {skog.max_features}")

```

Listing 19: Random forest-modell som tilpasser treningsdataene.

4b

For å teste modellen på valideringsdataene, brukte jeg følgende kode:

```
prediksjoner = skog.predict(X_val)
noyaktighet = skog.score(X_val, y_val)
print(f"Nøyaktighet: {np.round(noyaktighet, 4)}")
```

Listing 20: Testing av random forest-modell på valideringsdata.

Denne koden ga følgende output:

```
Nøyaktighet: 0.7821
```

Modellen hadde altså en treffprosent på om lag 78% når vi brukte 100 trær.

4c

For å undersøke hvordan antallet "weak learners" i modellen påvirker nøyaktigheten, tilpasset jeg modeller og regnet ut treffprosenten med følgende antall weak learners:

- 1
- 3
- 5
- 10
- 20
- 50
- 100

- 200
- 500
- 1000

Koden for å regne ut dette er gitt under:

```
weak_learners = [1, 3, 5, 10, 20, 50, 100, 200, 500, 1000]
accuracies = []
for n in weak_learners:
    rf = RandomForestClassifier(n_estimators = n, random_state=seed)
    rf.fit(X_train, y_train.ravel())
    accuracies.append(rf.score(X_val, y_val))
```

Listing 21: Utregning av accuracy score for ulike antall weak learners.

For å plotte resultatene brukte jeg følgende kode:

```
plt.plot(weak_learners, accuracies, marker="o")
plt.xlabel("Antall trær")
plt.ylabel("Testnøyaktighet")
plt.title("Random Forest: nøyaktighet vs. antall trær")
plt.xscale("symlog")
plt.show()

```

Listing 22: Plotting av accuracy score som funksjon av antall weak learners

I figur 4 ser vi at treffsikkerheten øker med økende antall weak learners, og at treffsikkerheten virker å nå et platå på omtrent 80% for når antallet weak learners er høyere enn 500.

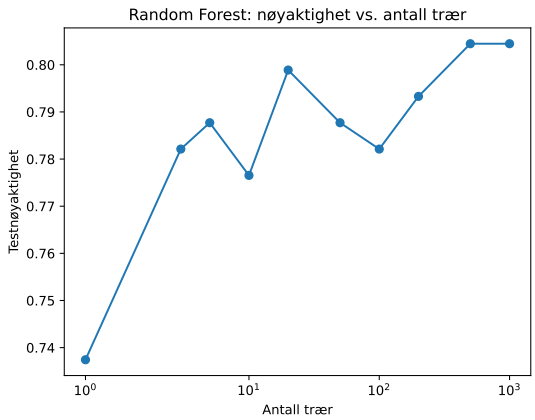

Figure 4: Treffsikkerhet som funksjon av antallet weak learners.

4d

Fra forrige oppgave så vi at vår beste modell hadde 500 weak learners. For å lage et plott over "feature importance" for denne modellen brukte jeg følgende kode:

```
n = 500
skog = RandomForestClassifier(n_estimators = n, random_state=seed)
skog.fit(X_train, y_train.ravel())
importance = skog.feature_importances_


plt.title("Feature Importances, Random Forest")
plt.barh(X_train.columns, importance, color="green")
plt.xlabel("Fetaure importance")
plt.savefig("../figures/feature_importance.pdf", format="pdf")

```

Listing 23: Feature importance for random forest-modell med 500 weak learners.

I Figur 5 ser man en oversikt over feature importance for alle forklaringsvariablene i modellen. Vi ser at de viktigste forklaringsvariablene i modellen er alder (`Age`) og billettpris (`Fare`), etterfulgt av kjønn (`Sex_female` og `Sex_male`).

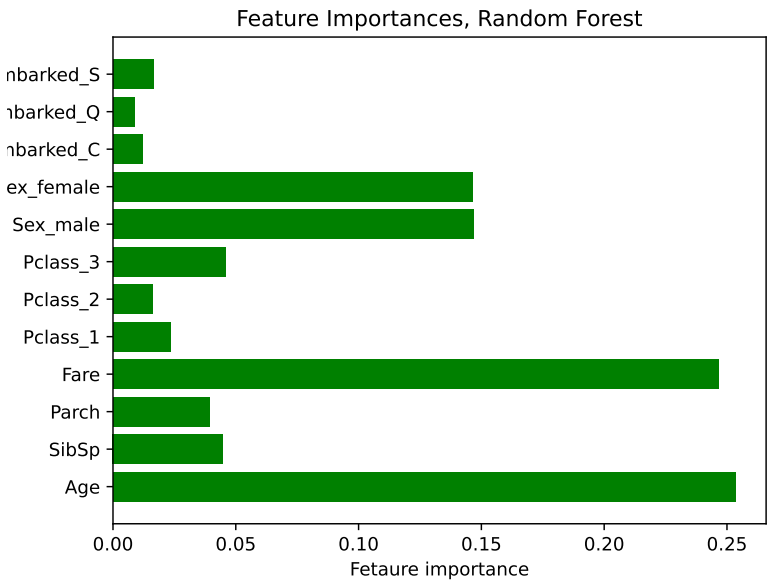

Figure 5: Feature importance for random forest-modell med 500 weak learners.

For å lage et plott av "partial dependece" for forklaringsvariablene brukte jeg koden under:

```python
from sklearn.inspection import PartialDependenceDisplay

# Kopier data
X_train_plot = X_train.copy()
X_val_plot = X_val.copy()

# Konverter numeriske kolonner til float
num_cols = X_train_plot.select_dtypes(include=[np.number]).columns
X_train_plot[num_cols] = X_train_plot[num_cols].astype(float)
X_val_plot[num_cols] = X_val_plot[num_cols].astype(float)


fig, ax = plt.subplots(figsize=(15, 12))
plt.title("Partial Dependence Display, alle forklaringsvariabler")
fig = PartialDependenceDisplay.from_estimator(skog, X_val_plot, features=X_train_plot.columns, ax=ax)

plt.tight_layout()
plt.savefig("../figures/partial_dependence_display.pdf", format="pdf")
plt.show()
```

Listing 24: Partial dependence plott for random forest-modell med 500 weak learners

I koden lager jeg en kopi av dataene og transformerer data av type `int` til å bli `float`, siden dette skapte advarselmeldinger fra scikit-learn.

I figur 6 ser vi de resulterende plottene. Vi ser at økende alder fører til en lavere sannsynlighet for å overleve, samtidig som økende billettpris fører til en høyere sannsynlighet for å overleve. Å være kvinne øker sannsynligheten for å overleve, mens å være mann minker sannsynligheten for å overleve. Billettklasse 1 øker også sannsynligheten for å overleve, mens sannsynligheten minker for billettklasse 3.

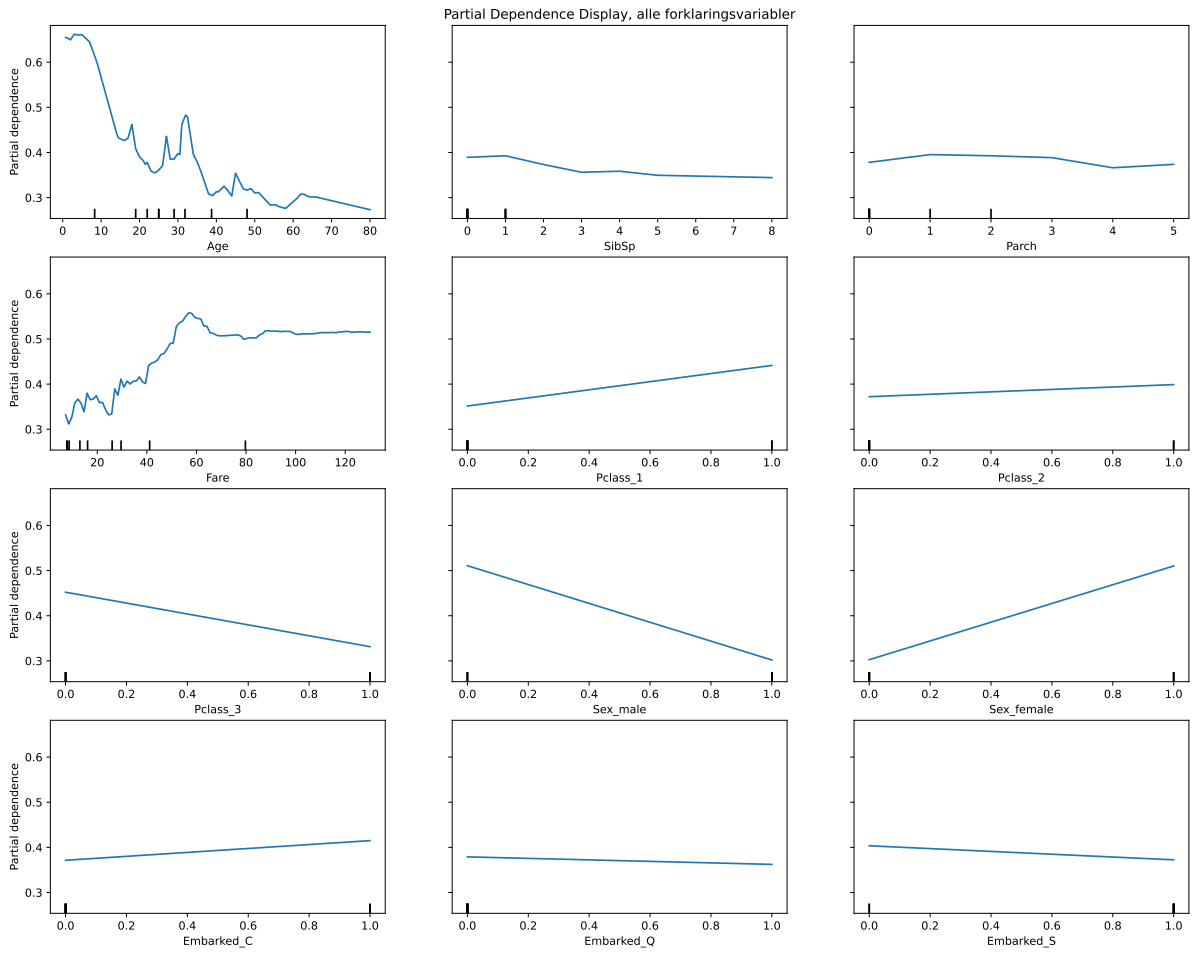

Figure 6: Partial dependence display for random forest-modell med 500 weak learners.

Fra plottene som viser feature importance og partial dependence ser vi at alder og billettpris er de viktigste faktorene for om noen overlevde på Titanic, etterfulgt av kjønn. Billettklasse påvirker også overlevelses-sannsynligheten, men disse forklaringsvariablene er mest sannsynlig svært korrelert med billettprisen.

4d

Det er ikke mulig vite nøyaktig hva random forest-modellen vil predikere for en spesifikk person, kun basert på feature importance og partial dependence. Feature importance måler hvor mye modellen har "lent seg" på hver forklaringsvariabel, men man kan ikke vite om forklaringsvariabler påvirker prediksjonen i positiv eller negativ retning. Partial dependence viser hvordan modellens prediksjoner endrer seg når man kun endrer en enkelt forklaringsvariabel, og holder alle andre forklaringsvariabler fast. Man kan altså få en ide om hvordan modellen har brukt forklaringsvariablene i prediksjonene, men man kan ikke vite nøyaktig hva modellen vil predikere for en gitt person. For random forest-modeller kan man forstå hvilke variabler som er viktigst når modellen gjør prediksjoner, men man kan ikke se nøyaktige beslutningsregler.

For beslutningstrær er situasjonen annerledes. Uavhengig av maksdybden til treet, kan man plotte en figur som viser hvilke beslutningsregler modellen bruker. Derfor kan man ta en spesifikk person, følge beslutningsreglene i treet, og vite hva modellen kommer til å predikere.

For en logistisk regresjonsmodell kan man også vite hva modellen vil predikere for en gitt input, så lenge man kjenner til koeffisientene i modellen. Under treningen av modellen blir koeffisientene estimert, og dersom man ganger disse koeffisientene med de korresponderende input-variablene, kan man vite hva modellen predikerer ved å bruke sigmoidfunksjonen på resultatet.

For å oppsummere, er logistisk regresjon og beslutningstrær modeller hvor man kan vite hvordan prediksjoner gjøres, mens random forest-modeller oppfører seg mer som en "black box", der man ikke

kan vite nøyaktig hva slags prediksjon man får for en gitt input.

Appendix

Implementering av logisisk regresjon

```python
import numpy as np

def sigmoid(z: float) -> float:
    """Sigmoid-funksjonen. Tar høyde for overflow med np.where()

    Args:
        z (float): Input

    Returns:
        float: Funksjonsverdi
    """
    return np.where (z >= 0, 1 / (1 + np.exp(-z)), np.exp(z) / (1 + np.exp(z)))

def vekter_bias(rad: int, kol: int, seed: int = None):
    """Lager vekter og biaser.

    Args:
        rad (int): Antall rader i designmatrisen
        kol (int): Antall kolonner i designmatrisen
        seed (int): Seed for reproduserbarhet. Optional

    Returns:
        Vekter W (matrise) og biaser b (skalar)
    """
    if seed is not None:
        rng = np.random.default_rng(seed)
        W = rng.random((kol, 1))
        b = rng.random()
    else:
        W = np.random.randn(kol, 1)
        b = np.random.randn()
    return W, b

def output(X: np.ndarray, W: np.ndarray, b: float) -> np.ndarray:
    """Ganger input med vekter og legger til bias. Kjører gjennom
    sigmoidfunksjonen

    Args:
        X (np.ndarray): Input X. Kan være treningsdata. Shape: (nxp)
        W (np.ndarray): Vekter. Shape: (px1)
        b (float): Bias (skalar)

    Returns:
        float: y_hat. Shape: (nx1)
    """
    z = X @ W + b
    y_hat = sigmoid(z)
    return y_hat
```

Listing 25: Egen implementering av logistisk regresjon, del 1

```python
def gradients(y: np.ndarray, y_hat: np.ndarray, X: np.ndarray):
    """Function for computing gradients.

    Args:
        y (np.ndarray): y. Shape nx1
        y_hat (np.ndarray): y_hat (output). Shape nx1
        X (np.ndarray): X. Shape nxp

    Returns:
        dCdW: gradient. Shape px1
        dCdb: gradient. Shape nx1
    """
    n = np.shape(y)[0]
    delta = - (y-y_hat) / n
    dCdW = X.T @ delta
    dCdb = np.sum(delta, axis=0)
    return dCdW, dCdb

def gradient_step(W: np.ndarray,
                  dCdW: np.ndarray,
                  b: np.ndarray,
                  dCdb: np.ndarray,
                  eta: float = 0.01):
    """Gjør et steg med gradient descent

    Args:
        W (np.ndarray): W før steg
        dCdW (np.ndarray): gradient W
        b (np.ndarray): b før steg
        dCdb (np.ndarray): gradient b
        eta (float, optional): Steglengde. Defaults to 0.01.

    Returns:
        W: W etter steg
        b: b etter steg
    """
    W = W - eta * dCdW
    b = b - eta * dCdb
    return W, b


```

Listing 26: Egen implementering av logistisk regresjon, del 2.

```python
def fit_model(X: np.ndarray,
              y: np.ndarray,
              seed: int = None,
              eta: float = 0.01,
              n_iters: int=1000):
    """Funksjon for å tilpasse modell med gradient descent. Data blir transformert
    til riktig type og shape. Printer for hver 10ende prosent i treningsløkka.

    Args:
        X (np.ndarray): Designmatrise
        y (np.ndarray): Respons
        seed (int, optional): Seed. Defaults to None.
        eta (float, optional): Læringsrate. Defaults to 0.01.
        n_iters (int, optional): Antall iterasjoner. Defaults to 1000.

    Returns:
        W: Vektor med vekter
        b: bias (skalar)
    """
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float).reshape(-1, 1)
    n, p = np.shape(X)
    W, b = vekter_bias(n, p, seed)

    step = n_iters // 10

    for i in range(n_iters):
        y_hat = output(X, W, b)
        dW, db = gradients(y, y_hat, X)
        W, b = gradient_step(W, dW, b, db, eta)
        if (i+1) % step == 0:
            print(f"Fullført {i+1} steg av {n_iters}")
    return W, b
```

Listing 27: Egen implementering av logistisk regresjon, del 3.